UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

Universidad del Perú, Decana de América

FACULTAD DE INGENIERÍA DE SISTEMAS E INFORMÁTICA

Escuela Profesional de Ciencias de la Computación



LENGUAJES Y COMPILADORES

FLOWXYRUST

Definición del lenguaje de programación

Curso:	Lenguajes y Compiladores
Profesor:	Ruiz De la Cruz Melo, Carlos Augusto
Integrantes:	Sotelo Atuncar, Diego Alejandro
	Rodriguez Claeyssen, Isaac Aaron
Fecha:	Septiembre de 2026

Lima – Perú

2026

Índice

Introducción **3**

Convención de sintaxis **3**

Elementos usados en las estructuras **3**

1. Estructura de un programa **5**

2. Identificadores **5**

2.1. Tipos de datos 6

2.2. Constantes 7

2.3. Variables 8

2.4. Vectores de una dimensión 8

2.5. Estructurados 10

3. Operadores **14**

3.1. Operadores aritméticos 16

3.2. Operadores relacionales 17

3.3. Operadores lógicos 17

3.4. Operadores de asignación 17

3.5. Prioridad de operadores 18

4. Expresiones **18**

4.1. Expresiones aritméticas 19

4.2. Expresiones relacionales 19

4.3. Expresiones lógicas 20

4.4. Expresiones de cadena de texto 20

5. Instrucciones **21**

5.1. Instrucciones declarativas 21

5.2. Instrucciones de asignación 21

5.3. Entrada y salida de datos 23

5.4. Retorno 23

6. Sentencias de control de flujo **23**

6.1. Si – sino 24

6.2. Mientras 24

6.3. Para 24

7. Subprogramas **25**

7.1. Parámetros y argumentos 25

7.2. Función sin retorno 26

7.3. Función con retorno 26

7.4. Métodos 27

7.5. Función principal 27

Resumen de estructuras **27**

Introducción

FlowxyRust es un lenguaje de programación inspirado en Rust. Se busca que tenga una sintaxis sencilla, ordenada y fácil de leer.

El lenguaje permitirá trabajar con tipos básicos, constantes, variables, vectores, clases, objetos, operadores, expresiones, instrucciones, sentencias de control y subprogramas.

En esta primera versión se utilizarán estructuras simples para facilitar su posterior análisis y traducción.

Convención de sintaxis

Para mostrar la forma general de cada estructura se utilizará la siguiente notación:

- <>: elemento obligatorio.
- `[ ]`: fragmento opcional.
- Los símbolos que aparecen fuera de estos elementos forman parte de la sintaxis real del lenguaje.

Ejemplo:

```
<sea>[<mutable>] <ID>: <TIPO_BASICO>[= <EXPRESION>];
```

En este caso, `sea`, el identificador y el tipo son obligatorios. El modificador `mutable` y la inicialización son opcionales.

Cuando los corchetes formen parte de un elemento obligatorio, serán símbolos reales del lenguaje. Por ejemplo:

```
<[TIPO_BASICO; TAMANIO]>
```

representa el tipo de un vector y los corchetes forman parte de su sintaxis.

Elementos usados en las estructuras

Para mantener la definición simple, se utilizarán los siguientes nombres:

Elemento	Representa
<ID>	Un identificador válido. Ejemplo: edad.
<TIPO_BASICO>	Un tipo definido directamente por FlowxyRust. Ejemplo: entero32.
<TIPO_ENTERO>	Un tipo entero con o sin signo. Ejemplo: entero32 o natural32.
<TIPO>	Un <TIPO_BASICO> o una clase definida por el programador. Ejemplo: entero32 o Persona.
<VALOR>	Un valor escrito directamente. Ejemplo: 14, 20.5, 'D', "Diego", verdadero.
<VALOR_VECTOR>	Un valor completo de vector. Su forma será [<VALORES>]. Ejemplo: [10, 20, 30].
<VALOR_OBJETO>	Un valor completo de objeto. Su forma será <CLASE>{ <VALORES_ATRIBUTOS>}. Ejemplo: Persona { nombre: "Diego", edad: 20 }.
<EXPRESION>	Una combinación válida que produce un valor. Ejemplo: a + b.
<OPERANDO>	Un elemento que puede participar en una expresión. Ejemplo: 10, edad o notas[0].
<DESTINO>	Un elemento de tipo básico al que se puede asignar un valor. Ejemplo: edad, notas[0] o persona.edad.
<INDICE>	Una expresión entera usada para acceder a una posición de un vector. Ejemplo: 0 o i.
<CONDICION>	Una expresión cuyo resultado es de tipo booleano. Ejemplo: edad >= 18.
<CLASE>	El identificador de una clase declarada. Ejemplo: Persona.
<OBJETO>	El identificador de un objeto declarado. Ejemplo: persona.
<ATRIBUTO>	El identificador de un atributo definido en una clase. Ejemplo: edad.
<METODO>	El identificador de un método definido para una clase. Ejemplo: obtenerEdad.
<ATRIBUTOS>	Uno o más atributos declarados dentro de una clase.
<METODOS>	Uno o más métodos declarados dentro de un bloque implementa.
<VALORES>	Uno o más valores separados mediante comas. Ejemplo: 10, 20, 30.
<SENTENCIAS>	Cero o más instrucciones válidas dentro de un bloque.

Los demás elementos como <PARAMETROS>, <ARGUMENTOS> o <VALORES_ATRIBUTOS> se explicarán en la sección donde sean utilizados.

Regla general de inicialización

Ninguna variable, vector u objeto podrá utilizarse antes de recibir un valor. Un elemento inmutable declarado sin valor podrá recibir una primera asignación y después no podrá modificarse. Un elemento declarado con mutable podrá modificarse después de haber sido inicializado.

Los vectores y objetos deberán inicializarse de forma completa.

En toda inicialización, asignación, argumento o retorno, el valor utilizado deberá ser compatible con el tipo esperado.

1. Estructura de un programa

Un programa FlowxyRust seguirá el siguiente orden:

Declaraciones globales → **Clases** → **Implementaciones** → **Funciones** → **mainfun**

Su forma general será:

```
[<DECLARACIONES_GLOBALES>] [<CLASES>] [<IMPLEMENTACIONES>]
                    [<FUNCIONES>] <MAINFUN>
```

Los cuatro primeros bloques serán opcionales. Todo programa deberá tener una única función principal `mainfun`.

Las declaraciones globales podrán corresponder a constantes, variables y vectores. Un objeto solo podrá declararse después de que su clase haya sido definida.

La función principal tendrá la forma:

```
<mainfun>() { <SENTENCIAS>}
```

Ejemplo válido:

```
mainfun() {
    sea edad : entero32 = 20;
    mostrar(edad);
}
```

2. Identificadores

Un identificador será el nombre utilizado para reconocer variables, constantes, vectores, clases, atributos, métodos y funciones.

Las reglas serán:

- Debe comenzar con una letra entre `A-Z` o `a-z`.
- Después podrá contener letras, números o _.
- No podrá comenzar ni terminar con _.
- No podrá contener __.
- No se utilizarán tildes ni la letra ñ.
- Se diferenciarán mayúsculas y minúsculas.

- Una palabra reservada no podrá utilizarse como identificador.

Ejemplos válidos:

```
edad
persona1
numero_total
funPrincipal
```

Ejemplos inválidos:

```
_edad
edad_
numero__total
2persona
mi-edad
```

Palabras reservadas

Categoría	Palabras reservadas
Declaración	`sea, constante`
Modificador	`mutable`
Orientación a objetos	`clase, implementa, este`
Tipos enteros	`entero16, entero32, entero64, natural16, natural32, natural64`
Tipos flotantes	`flotante16, flotante32, flotante64`
Otros tipos	`caracter, cadena, booleano`
Valores booleanos	`verdadero, falso`
Entrada y salida	`leer, mostrar`
Control de flujo	`si, sino, mientras, para`
Subprogramas	`fun, retornar, mainfun`

2.1. Tipos de datos

FlowxyRust tendrá tipos básicos y tipos estructurados.

Tipos básicos

Tipo	Representa
`entero16 / entero32 / entero64`	Números enteros con signo.
`natural16 / natural32 / natural64`	Números enteros sin signo.
`flotante16 / flotante32 / flotante64`	Números decimales.
`caracter`	Un carácter escrito entre comillas simples. Ejemplo: `'D'`.
`cadena`	Texto escrito entre comillas dobles. Ejemplo: `"Diego"`.
`booleano`	Un valor `verdadero` o `falso`.

El nombre de una clase será considerado un tipo estructurado.

Un <VALOR> será un valor escrito directamente en el programa. Los valores numéricos podrán escribirse con signo negativo, por ejemplo `-10`; posteriormente el signo `-` será tratado como un operador unario.

2.2. Constantes

Una constante tendrá un valor fijo y deberá inicializarse al ser declarada.

```
<constante><ID>: <TIPO_BASICO>= <VALOR>;
```

Ejemplos válidos:

```
constante MAXIMO : entero32 = 100;
constante pi : flotante64 = 3.141592;
constante ACTIVO : booleano = verdadero;
```

Se podrá utilizar cualquier identificador válido para una constante, pero se recomienda escribirlo en mayúsculas.

Ejemplo recomendado:

```
constante MAXIMO : entero32 = 100;
```

Ejemplo válido, pero no recomendado:

```
constante maximo : entero32 = 100;
```

Ejemplo inválido:

```
constante LIMITE : entero32;
```

Una constante siempre deberá tener un valor al momento de su declaración.

2.3. Variables

Las variables se declararán con la palabra `sea`. Por defecto serán inmutables. El modificador `mutable` permitirá cambiar su valor después de haber sido inicializada.

```
<sea>[<mutable>] <ID>: <TIPO_BASICO>[= <EXPRESION>];
```

Ejemplos válidos:

```
sea edad : entero32 = 20;
sea nombre : cadena;
sea mutable contador : entero32 = 0;
```

Una variable declarada sin valor existirá, pero no podrá utilizarse hasta recibir su primera asignación.

Ejemplo válido:

```
sea edad : entero32;
edad = 20;
mostrar(edad);
```

Ejemplo inválido:

```
sea edad : entero32;
mostrar(edad);
```

Una variable inmutable declarada sin valor podrá recibir una primera asignación. Después de ello no podrá modificarse.

```
sea edad : entero32;
edad = 20;      // valido
edad = 21;      // invalido
```

Una variable declarada con `mutable` sí podrá modificarse varias veces.

```
sea mutable edad : entero32 = 20;
edad = 21;
edad += 1;
```

2.4. Vectores de una dimensión

Los vectores serán de una dimensión, tendrán elementos del mismo tipo y su tamaño será fijo.

```
<sea>[<mutable>] <ID>: <[TIPO_BASICO; TAMANIO]>[= <VALOR_VECTOR>];
```

<TAMANIO> será un número entero positivo escrito directamente. <VALOR_VECTOR> tendrá la forma:

[<VALORES>]

<VALORES> representa valores del mismo tipo separados mediante comas. La cantidad de valores deberá coincidir con <TAMANIO>.

En <VALOR_VECTOR>, los símbolos [y] son obligatorios y forman parte de la sintaxis del lenguaje.

Ejemplo:

```
sea mutable notas : [flotante32; 3] = [15.5, 18.0, 14.5];
```

Un vector declarado sin valores no podrá utilizarse hasta recibir una asignación completa.

La asignación completa tendrá la forma:

<ID>= <VALOR_VECTOR>;

Ejemplo válido:

```
sea mutable notas : [flotante32; 3];
notas = [15.5, 18.0, 14.5];
mostrar(notas[0]);
```

Ejemplos inválidos:

```
sea mutable notas : [flotante32; 3];
mostrar(notas[0]);

sea notas : [entero32; 3] = [10, 20];
```

En el segundo caso faltaría un valor porque el tamaño declarado es 3.

El acceso a una posición tendrá la forma:

<ID>[<INDICE>]

Los índices comenzarán en cero. El índice deberá producir un valor entero y estar dentro del tamaño del vector.

Ejemplo:

```
notas[0]
notas[2]
```

Un vector inicializado podrá consultarse aunque sea inmutable. Sin embargo, una posición solo podrá modificarse si el vector fue declarado con `mutable`.

Ejemplo válido:

```
sea mutable notas : [entero32; 3] = [10, 15, 20];
notas[1] = 18;
```

Ejemplo inválido:

```
sea notas : [entero32; 3] = [10, 15, 20];
notas[1] = 18;
```

2.5. Estructurados

Las clases permitirán definir tipos creados por el programador.

Clase

La forma de una clase será:

<clase><ID>{ <ATRIBUTOS>}

Se podrá utilizar cualquier identificador válido como nombre de clase, pero se recomienda comenzar con una letra mayúscula.

Ejemplo recomendado:

```
clase Persona {
    sea nombre : cadena;
    sea edad : entero32;
}
```

Ejemplo válido, pero no recomendado:

```
clase persona {
    sea nombre : cadena;
}
```

Atributo

Un atributo tendrá un tipo básico.

<sea><ID>: <TIPO_BASICO>;

Ejemplo:

```
sea edad : entero32;
```

Los atributos indican su nombre y tipo. Su valor será proporcionado al crear un objeto.

Implementación

Los métodos de una clase se colocarán dentro de un bloque `implementa`.

<implementa><CLASE>{ <METODOS>}

<CLASE> representa el nombre de una clase declarada anteriormente y <METODOS> representa uno o más métodos.

Cada clase podrá tener como máximo un bloque `implementa`. Si una clase no tiene métodos, este bloque podrá omitirse.

Ejemplo:

```
implementa Persona {

    fun verEdad() {
        mostrar(este.edad);
    }
}
```

Método

Los métodos serán funciones asociadas a una clase y se declararán dentro de un bloque `implementa`.

La forma general de un método será:

```
<fun>[<mutable>] <ID>([<PARAMETROS>]) [<TIPO_RETORNO>] { <SENTENCIAS>}
```

El elemento <TIPO_RETORNO> será opcional e indicará el tipo de valor que devuelve el método.

Su forma será:

```
: <TIPO_BASICO>
```

Por lo tanto, un método podrá ser de dos formas.

Método sin retorno:

```
<fun>[<mutable>] <ID>([<PARAMETROS>]) { <SENTENCIAS>}
```

Ejemplo:

```
fun mostrarEdad() {
    mostrar(este.edad);
}
```

Un método sin retorno no devolverá ningún valor.

Método con retorno:

```
<fun>[<mutable>] <ID>([<PARAMETROS>]) : <TIPO_BASICO>{ <SENTENCIAS>}
```

Un método con retorno deberá devolver un valor mediante la instrucción:

```
<retornar><EXPRESION>;
```

La expresión retornada deberá ser compatible con el tipo indicado en <TIPO_RETORNO>.

Ejemplo:

```
fun obtenerEdad() : entero32 {
    retornar este.edad;
}
```

En este caso:

```
: entero32
```

indica que el método devuelve un valor de tipo `entero32`, y:

```
retornar este.edad;
```

es la instrucción que devuelve dicho valor.

La llamada a un método sin retorno se utilizará como una instrucción:

```
<OBJETO>.<METODO>([<ARGUMENTOS>]);
```

Ejemplo:

```
persona.mostrarEdad();
```

La llamada a un método con retorno podrá utilizarse como parte de una expresión:

```
<OBJETO>.<METODO>([<ARGUMENTOS>])
```

Ejemplo:

```
sea edad : entero32 = persona.obtenerEdad();
```

El modificador `mutable` indicará que el método puede modificar los atributos del objeto actual.

Ejemplo:

```
fun mutable cumplirAnios() {
    este.edad = este.edad + 1;
}
```

También podrá existir un método `mutable` con retorno:

```
fun mutable cumplirYObtenerEdad() : entero32 {
    este.edad = este.edad + 1;
    retornar este.edad;
}
```

Un método `mutable` solo podrá llamarse desde un objeto declarado también como `mutable`.

La palabra `este` solo podrá utilizarse dentro de un método.

Objeto

Un objeto será una variable cuyo tipo será una clase.

La declaración tendrá la forma:

<sea>[<mutable>] <ID>: <CLASE>[= <VALOR_OBJETO>];

<VALOR_OBJETO> tendrá la forma:

<CLASE>{ <VALORES_ATRIBUTOS>}

Cada valor de atributo tendrá la forma:

<ATRIBUTO>: <EXPRESION>

<VALORES_ATRIBUTOS> representa los valores de los atributos separados por comas. Deberán aparecer todos los atributos, una sola vez, con valores compatibles y siguiendo el mismo orden en que fueron declarados en la clase.

Ejemplo:

```
sea mutable persona : Persona = Persona {
    nombre: "Diego",
    edad: 20
};
```

Un objeto declarado sin valor no podrá utilizarse hasta recibir una asignación completa.

La asignación completa tendrá la forma:

<OBJETO>= <VALOR_OBJETO>;

Ejemplo válido:

```
sea persona : Persona;

persona = Persona {
    nombre: "Diego",
    edad: 20
};
```

Ejemplos inválidos:

```
sea persona : Persona;
mostrar(persona.edad);

sea otra : Persona = Persona {
    nombre: "Ana"
};
```

En el segundo caso falta el atributo `edad`.

El acceso a un atributo será:

```
<OBJETO>.<ATRIBUTO>
```

Ejemplo:

```
persona.edad
```

La llamada a un método como instrucción será:

```
<OBJETO>.<METODO>([<ARGUMENTOS>]);
```

Si el método retorna un valor, su llamada también podrá utilizarse como parte de una expresión, en cuyo caso no lleva `;` dentro de la expresión:

```
<OBJETO>.<METODO>([<ARGUMENTOS>])
```

Dentro de un método también podrá utilizarse `este`:

```
<este>.<ATRIBUTO>
```

```
<este>.<METODO>([<ARGUMENTOS>])
```

Un atributo de un objeto solo podrá modificarse si el objeto fue declarado con `mutable`.

Un objeto solamente podrá acceder a los atributos y llamar a los métodos definidos para su propia clase.

3. Operadores

Un operador es un símbolo que actúa sobre uno o más operandos.

Según la posición y cantidad de operandos, FlowxyRust tendrá operadores unarios prefijos, binarios y unarios postfijos.

Operador unario prefijo

Un operador unario prefijo actúa sobre un solo operando y se escribe antes de este.

Su forma será:

```
<OPERADOR_UNARIO><OPERANDO>
```

Los operadores unarios serán:

```
-     !
```

Ejemplos:

```
-3
-temperatura
!activo
```

El operador `-` negará un valor numérico y el operador `!` negará un valor de tipo `booleano`.

Operador binario

Un operador binario actúa sobre dos operandos y se escribe entre ellos.

Su forma será:

<OPERANDO><OPERADOR_BINARIO><OPERANDO>

Ejemplos:

```
a + b
edad >= 18
activo && registrado
```

El elemento <OPERADOR_BINARIO> podrá representar un operador aritmético, relacional o lógico.

Operador unario postfijo

Un operador postfijo actúa sobre un solo destino y se escribe después de este.

Su forma será:

<DESTINO><OPERADOR_POSTFIJO>

Los operadores postfijos serán:

++ --

Ejemplos:

```
contador++
contador--
```

Los operadores postfijos se utilizarán sobre destinos numéricos inicializados y modificables.

Categorías de operadores

Para las estructuras del lenguaje se utilizarán las siguientes categorías.

Operador unario

<OPERADOR_UNARIO>

\- !

Operador postfijo

<OPERADOR_POSTFIJO>

```
++      --
```

Operador aritmético

```
<OPERADOR_ARITMETICO>

+      -      *      / %
```

Operador relacional

```
<OPERADOR_RELACIONAL>

==      !=      <      >      <=      >=
```

Operador lógico

```
<OPERADOR_LOGICO>

&&      ||
```

Operador de asignación

```
<OPERADOR_ASIGNACION>

=      +=      -=      *=      /= %=
```

3.1. Operadores aritméticos

Los operadores aritméticos trabajarán principalmente con valores numéricos.

Operador	Uso	Ejemplo
+	Suma	`a + b`
-	Resta	`a - b`
*	Multiplicación	`a * b`
/	División	`a / b`
%	Módulo	`a% b`
++	Incremento	`a++`
--	Decremento	`a--`

El operador - también podrá utilizarse como operador unario para negar un valor numérico.

```
<OPERADOR_UNARIO><OPERANDO>
```

Ejemplo:

```
-3
-temperatura
```

El operador + también podrá concatenar dos valores de tipo `cadena`.

3.2. Operadores relacionales

Los operadores relacionales compararán dos valores y producirán un resultado de tipo `booleano`.

Operador	Uso	Ejemplo
==	Igual	`a == b`
!=	Diferente	`a != b`
<	Menor	`a <b`
>	Mayor	`a >b`
<=	Menor o igual	`a <= b`
>=	Mayor o igual	`a >= b`

Los operadores == y `!=` podrán comparar valores de tipos compatibles. Los demás operadores relacionales se utilizarán con valores numéricos.

3.3. Operadores lógicos

Los operadores lógicos trabajarán con valores de tipo `booleano`.

Operador	Uso	Ejemplo
&&	AND	`a && b`
`||`	OR	`a || b`
!	Negación	`!a`

&& y `||` serán operadores binarios. El operador `!` será unario prefijo.

3.4. Operadores de asignación

Los operadores de asignación se utilizarán dentro de instrucciones de asignación.

Operador	Uso
=	Asignación
+=	Suma y asignación
-=	Resta y asignación
*=	Multiplicación y asignación
/=	División y asignación
%=	Módulo y asignación

El operador = podrá utilizarse con destinos compatibles. Los operadores +=, -=, *=, /= y %= se utilizarán sobre destinos numéricos modificables.

3.5. Prioridad de operadores

La prioridad indica el orden en que se evaluarán los operadores dentro de una expresión.

Los paréntesis `()` tendrán la mayor prioridad y permitirán agrupar una expresión para indicar que debe evaluarse primero.

Los accesos a atributos, la indexación de vectores y las llamadas a funciones o métodos también se resolverán antes de aplicar los operadores restantes.

Prioridad	Operadores o estructura
1	`()`
2	`!` `-`
3	`*` `/` `%`
4	`+` `-`
5	`<` `<=` `>` `>=`
6	`==` `!=`
7	`&&`
8	`||`

El nivel 1 representa la prioridad más alta.

Ejemplo:

```
a + b * c
```

En este caso primero se evaluará:

```
b * c
```

Si se utilizan paréntesis:

```
(a + b) * c
```

primero se evaluará:

```
a + b
```

Cuando dos operadores binarios tengan la misma prioridad, se evaluarán de izquierda a derecha.

Los operadores `++`, `--` y los operadores de asignación no se incluirán en esta tabla porque se utilizarán como instrucciones y no como partes de expresiones compuestas.

4. Expresiones

Una expresión será una combinación de operandos y operadores que produce un resultado. Una expresión podrá contener otras expresiones respetando la prioridad definida anteriormente.

Un `<OPERANDO>` podrá ser:

Elemento	Forma
Valor	<VALOR>
Variable o constante	<ID>
Elemento de vector	<ID>[<INDICE>]
Atributo	<OBJETO>.<ATRIBUTO>
Atributo actual	<este>.<ATRIBUTO>
Llamada con retorno	<ID>([<ARGUMENTOS>])
Método con retorno	<OBJETO>.<METODO>([<ARGUMENTOS>])
Método actual con retorno	<este>.<METODO>([<ARGUMENTOS>])
Expresión agrupada	(<EXPRESION>)

Todo operando basado en una variable, vector u objeto deberá haber sido inicializado previamente.

4.1. Expresiones aritméticas

La forma binaria será:

<OPERANDO><OPERADOR_ARITMETICO><OPERANDO>

La forma unaria será:

<OPERADOR_UNARIO><OPERANDO>

En una expresión aritmética, <OPERADOR_UNARIO> corresponderá únicamente al operador -.

Ejemplos válidos:

```
a + b
precio * cantidad
numero % 2
-3
(a + b) * 2
```

4.2. Expresiones relacionales

La forma será:

<OPERANDO><OPERADOR_RELACIONAL><OPERANDO>

Ejemplos:

```
edad >= 18
a == b
numero != 0
```

El resultado será de tipo booleano.

4.3. Expresiones lógicas

La forma binaria será:

<EXPRESION><OPERADOR_LOGICO><EXPRESION>

La negación tendrá la forma:

!<EXPRESION>

Ejemplos válidos:

```
edad >= 18 && activo
autorizado || administrador
!activo
(edad >= 18) && activo
```

4.4. Expresiones de cadena de texto

El operador + permitirá concatenar dos expresiones de tipo `cadena`.

<EXPRESION_CADENA>+ <EXPRESION_CADENA>

Ejemplos:

```
"Hola " + nombre
nombre + " Sotelo"
```

Agrupación de expresiones

Una expresión podrá agruparse usando paréntesis.

(<EXPRESION>)

Ejemplo:

```
(a + b) * c
```

Comentarios

Los comentarios serán de una sola línea.

// <COMENTARIO>

Todo lo escrito después de `//` hasta el final de la línea será ignorado.

Ejemplo:

```
// Declaracion de edad
sea edad : entero32 = 20;
```

5. Instrucciones

FlowxyRust tendrá instrucciones declarativas, de asignación, de entrada y salida, de retorno, selectivas y repetitivas.

Las instrucciones selectivas y repetitivas se definirán en la sección de control de flujo.

5.1. Instrucciones declarativas

Las instrucciones declarativas crean un elemento con un nombre y un tipo.

Elemento	`Estructura`
Constante	`<constante><ID>: <TIPO_BASICO>= <VALOR>;`
Variable	`<sea>[<mutable>] <ID>: <TIPO_BASICO>[= <EXPRESION>];`
Vector	`<sea>[<mutable>] <ID>: <[TIPO_BASICO; TAMANIO]>[= <VALOR_VECTOR>];`
Objeto	`<sea>[<mutable>] <ID>: <CLASE>[= <VALOR_OBJETO>];`

Declarar crea el elemento. Inicializar significa darle su primer valor. Un elemento sin valor no podrá ser utilizado hasta ser inicializado.

5.2. Instrucciones de asignación

Una asignación guarda un valor en un destino.

Para variables básicas, posiciones de vector y atributos, la forma general será:

`<DESTINO><OPERADOR_ASIGNACION><EXPRESION>;`

La asignación completa de un vector será:

`<ID>= <VALOR_VECTOR>;`

La asignación completa de un objeto será:

`<OBJETO>= <VALOR_OBJETO>;`

Un `<DESTINO>` podrá ser:

Destino	**Forma**
Variable	`<ID>`
Elemento de vector	`<ID>[<INDICE>]`
Atributo de objeto	`<OBJETO>.<ATRIBUTO>`
Atributo actual	`<este>.<ATRIBUTO>`

Ejemplos válidos:

```
edad = 20;
contador += 1;
notas[0] = 18.5;
persona.edad = 21;

notas = [10, 15, 20];

persona = Persona {
    nombre: "Diego",
    edad: 20
};
```

Dentro de un método mutable también será válido modificar un atributo mediante `este`:

```
fun mutable cambiarEdad() {
    este.edad = 22;
}
```

El valor asignado deberá ser compatible con el tipo del destino.

Si un elemento no tiene valor, una asignación con = podrá darle su primer valor. Para modificar posteriormente un valor será necesario que el elemento sea `mutable`.

Los operadores compuestos se utilizarán solamente cuando el destino numérico ya tenga un valor.

Ejemplo inválido:

```
sea mutable contador : entero32;
contador += 1;
```

Primero deberá recibir un valor:

```
sea mutable contador : entero32;
contador = 0;
contador += 1;
```

El incremento y decremento tendrán las formas:

```
<DESTINO>++;     <DESTINO>--;
```

Ejemplos:

```
contador++;
contador--;
```

Solo se aplicarán a destinos numéricos inicializados y modificables.

5.3. Entrada y salida de datos

La entrada de datos tendrá la forma:

<leer>(<DESTINO>);

`leer` asignará el valor ingresado al destino. Podrá utilizarse para realizar la primera asignación de un elemento sin valor. Si el elemento ya tiene un valor, deberá ser `mutable`.

La salida tendrá la forma:

<mostrar>(<EXPRESION>);

La expresión utilizada en `mostrar` deberá contener únicamente elementos ya inicializados.

Ejemplos válidos:

```
sea edad : entero32;
leer(edad);
mostrar(edad);

mostrar("Hola " + nombre);
```

Ejemplo inválido:

```
sea edad : entero32;
mostrar(edad);
```

5.4. Retorno

La instrucción de retorno tendrá la forma:

<retornar>[<EXPRESION>];

Una función sin retorno podrá utilizar:

```
retornar;
```

Una función con retorno utilizará una expresión compatible con su tipo:

```
retornar resultado;
retornar a + b;
```

6. Sentencias de control de flujo

FlowxyRust tendrá instrucciones selectivas y repetitivas. En todas ellas <CONDICION> deberá producir un valor de tipo `booleano`.

6.1. Si – sino

La estructura será:

```
<si>(<CONDICION>) {
    <SENTENCIAS>
}
[<sino>{
    <SENTENCIAS>
}]
```

El bloque `sino` será opcional.

Ejemplo válido:

```
si (edad >= 18) {
    mostrar("Mayor de edad");
}
sino {
    mostrar("Menor de edad");
}
```

Ejemplo inválido:

```
si (20) {
    mostrar("Hola");
}
```

La condición debe ser booleana.

6.2. Mientras

La estructura será:

```
<mientras>(<CONDICION>) {
    <SENTENCIAS>
}
```

Ejemplo:

```
mientras (contador < 5) {
    contador++;
}
```

6.3. Para

La estructura será:

```
<para>(<INICIALIZACION>; <CONDICION>; <ACTUALIZACION>) { <SENTENCIAS>}
```

<INICIALIZACION> establece el valor inicial de la variable de control. Podrá tener una de las siguientes formas:

<sea><mutable><ID>: <TIPO_ENTERO>= <EXPRESION>

o:

<ID>= <EXPRESION>

Cuando se utilice una variable declarada previamente, deberá ser de tipo entero y haber sido declarada con `mutable`.

<ACTUALIZACION> modifica la variable de control después de cada repetición. Podrá tener la forma:

<ID><OPERADOR_POSTFIJO>

o:

<ID><OPERADOR_ASIGNACION><EXPRESION>

Los signos `;` de la estructura de `para` separarán la inicialización, la condición y la actualización.

Ejemplo válido:

```
para (sea mutable i : entero32 = 0; i < 5; i++) {
    mostrar(i);
}
```

7. Subprogramas

Los subprogramas se declararán mediante la palabra `fun`.

En esta primera versión, los parámetros y los valores de retorno utilizarán únicamente tipos básicos.

7.1. Parámetros y argumentos

Un parámetro tendrá la forma:

<ID>: <TIPO_BASICO>

<PARAMETROS> representa uno o más parámetros separados mediante comas.

Ejemplo:

```
a : entero32, b : entero32
```

<ARGUMENTOS> representa una o más expresiones enviadas durante una llamada, separadas mediante comas.

Ejemplo:

```
sumar(10, 20)
```

Los argumentos deberán corresponder con la cantidad y los tipos de los parámetros.

7.2. Función sin retorno

La estructura será:

```
<fun><ID>([<PARAMETROS>]) {
    <SENTENCIAS>
}
```

Ejemplo:

```
fun saludar(nombre : cadena) {
    mostrar("Hola " + nombre);
}
```

La llamada a una función sin retorno tendrá la forma:

```
<ID>([<ARGUMENTOS>]);
```

Ejemplo:

```
saludar("Diego");
```

7.3. Función con retorno

La estructura será:

```
<fun><ID>([<PARAMETROS>]) : <TIPO_BASICO>{
    <SENTENCIAS>
}
```

Ejemplo:

```
fun sumar(a : entero32, b : entero32) : entero32 {
    retornar a + b;
}
```

Una función con retorno deberá devolver un valor mediante `retornar`. El valor retornado deberá ser compatible con el tipo de retorno indicado.

Una llamada a una función con retorno podrá utilizarse dentro de una expresión. En este caso, la llamada no lleva `;` dentro de la expresión.

Ejemplo:

```
sea resultado : entero32 = sumar(4, 6);
```

7.4. Métodos

Los métodos seguirán la misma estructura de las funciones, pero estarán dentro de un bloque `implementa`.

```
<fun>[<mutable>] <ID>([<PARAMETROS>]) [<RETORNO>] { <SENTENCIAS>}
```

El elemento <RETORNO> representa:

```
: <TIPO_BASICO>
```

Cuando un método indique un tipo de retorno, deberá devolver un valor compatible mediante `retornar`.

Ejemplo:

```
fun obtenerEdad() : entero32 {
    retornar este.edad;
}
```

7.5. Función principal

La función principal será única, no tendrá parámetros ni tipo de retorno.

```
<mainfun>() { <SENTENCIAS>}
```

Ejemplo:

```
mainfun() {
    sea resultado : entero32 = 10;
    mostrar(resultado);
}
```

Resumen de estructuras

En la siguiente tabla se muestran las principales estructuras definidas para FlowxyRust.

Elemento	Estructura general
Programa	[<DECLARACIONES_GLOBALES>] [<CLASES>] [<IMPLEMENTACIONES>] [<FUNCIONES>] <MAINFUN>
Constante	<constante><ID>: <TIPO_BASICO>= <VALOR>;
Variable	<sea>[<mutable>] <ID>: <TIPO_BASICO>[= <EXPRESION>];
Vector	<sea>[<mutable>] <ID>: <[TIPO_BASICO; TAMANIO]>[= <VALOR_VECTOR>];
Clase	<clase><ID>{ <ATRIBUTOS>}
Implementación	<implementa><CLASE>{ <METODOS>}

Elemento	Estructura general
Método	<fun>[<mutable>] <ID>([<PARAMETROS>]) [<TIPO_RETORNO>] { <SENTENCIAS>}
Objeto	<sea>[<mutable>] <ID>: <CLASE>[= <VALOR_OBJETO>];
Expresión	<OPERANDO><OPERADOR><OPERANDO>
Asignación	<DESTINO><OPERADOR_ASIGNACION><EXPRESION>;
Entrada	<leer>(<DESTINO>);
Salida	<mostrar>(<EXPRESION>);
Retorno	<retornar>[<EXPRESION>];
Si – sino	<si>(<CONDICION>) { <SENTENCIAS>} [<sino>{ <SENTENCIAS>}]
Mientras	<mientras>(<CONDICION>) { <SENTENCIAS>}
Para	<para>(<INICIALIZACION>; <CONDICION>; <ACTUALIZACION>) { <SENTENCIAS>}
Función	<fun><ID>([<PARAMETROS>]) [<TIPO_RETORNO>] { <SENTENCIAS>}
Función principal	<mainfun>() { <SENTENCIAS>}
Comentario	// <COMENTARIO>